

50.043 Database Systems

Project Lab 1 Report

Class: CI04

Names: Tan Li Hon (1008920) & Celeste Chan Shi Ting (1008994)

Design Decisions

4 Bidirectional indexing with Thread Safety | [Catalog.java \(lines 30-33\)](#)

```
30     private ConcurrentHashMap<Integer, DbFile> idToFile;  
31     private ConcurrentHashMap<Integer, String> idToName;  
32     private ConcurrentHashMap<Integer, String> idToPkey;  
33     private ConcurrentHashMap<String, Integer> nameToId;
```

Catalog contains four `ConcurrentHashMap` for $O(1)$ lookup by hashing. Lines 30-31 serves the search by id queries to File, Name and PKey. Line 33 is a reverse index added so that name to id (`getTableId`) is also $O(1)$ instead of $O(n)$ scan of id-key map values. If a duplicated table name is added, the old entry will be removed from each map before inserting a new one to prevent stale mappings. `ConcurrentHashMap` is used over `HashMap` to honor the class's `@Threadsafe` contract.

Table id from the file path | [HeapFile.java \(line 63\)](#)

`HeapFile.getId()` returns `file.getAbsolutePath().hashCode()` in line 63. Hashing is used to create unique id per tableid, while ensuring stable tableid. This ensures the same file yields the same id on every run.

All reads go through the BufferPool | [HeapFile.java \(lines 202-204\)](#)

```
202         HeapPageId hpid = new HeapPageId(getId(), pageNo);  
203         HeapPage page = (HeapPage) Database.getBufferPool().getPage(tid, hpid, Permissions.READ_ONLY);  
204         return page.iterator();
```

The `HeapFile` iterator fetches the pages using `.getPage()` as seen in line 203, instead of calling `.readPage()` directly. This ensures that reads are cached. Calling `.readPage()` would bypass the buffer pool, avoiding repetitive expensive disk reads and decoding.

Bitmap header for slot tracking | [HeapPage.java](#)

Every `HeapPage` carries a header bitmap with one bit per tuple slot. Given that slot widths are fixed, desired slot can be predicted by byte offset, giving $O(1)$ addressing and making a insert/delete a single bit flip with a tradeoff of empty slots still occupying space on disk.

Consistent equals/hashCode for TupleDesc | [TupleDesc.java \(lines 205-214\)](#)

Our `TupleDesc.equals()` defines two schemas as equal when they have the same field types in the same order. In order to honor the `equals/hashCode` contract where equal objects must return equal hash codes, we implemented `hashCode()` to combine the field types in order, using standard $31 * \text{result} + \text{element}$.

Hashing over types guarantees that any two `TupleDesc` deemed equal by `equals()` also produces the same hash. This makes `TupleDesc` safe to use as a key in hash collections, where inconsistent hash codes would otherwise cause equal schemas to be slotted to different buckets and lookups will fail

Non-Trivial Code

Page Capacity | **HeapPage.java**

```
74  ✓    private int getNumTuples() {  
75      // some code goes here  
76      return (int) Math.floor((BufferPool.getPageSize() * 8.0) / (td.getSize() * 8 + 1));
```

To keep consistent calculations, all calculation will be done in bits by multiplying by 8. Each tuple costs `td.getSize()*8` plus 1 for header bit. Including that header bit into tuple size in denominator avoids circular dependency by charging each tuple its own header bit as it is counted. The decimal 8.0 forces floating point division so that fractional part survives to `Math.floor`. Its companion `getHeaderSize()` in line 87 uses `Math.ceil( getNumTuples() / 8.0 )` for opposite rounding as a partial byte must round up to cover every slot.

Reading one occupancy bit | **HeapPage.java (lines 303-305)**

```
303      int byteIndex = i / 8;  
304      int bitIndex = i % 8;  
305      return ((header[byteIndex] >> bitIndex) & 1) == 1;
```

`i/8` locate the byte, `i%8` locate the bit within that byte. `>> bitIndex` slides the target bit down to position 0 (left most), `&1` masks away every other bit. If it returns 1, that slot is taken. If it returns 0, that slot is empty.

Two level iteration with empty page skipping | **HeapFile.java (lines 172-183)**

```
172      while (!tupleIterator.hasNext())  
173      {  
174          currentPageNo++;  
175          if (currentPageNo >= numPages())  
176          {  
177              return null;  
178          }  
179          tupleIterator = getTupleIteratorForPage(currentPageNo);  
180      }  
181      return tupleIterator.next();  
182  }  
183  }
```

`HeapFileIterator()` extends `AbstractDBFileIterator` so only `readNext()` is implemented. The parent provides `hasNext()` and `next()`. The outer loop advances across pages while the inner loop iterates over the page's tuples. Outer loop uses "while" instead of "if" to account for consecutive empty pages, as an if-loop can only skip one empty page and return a stale "no next". `close()` nulls the tuple iterator so that `readNext()` returns null. This is to prevent a post `close next()` call from returning a tuple instead of throwing `NoSuchElementException` to signal exhaustion.

Staying byte-aligned over empty slots | **HeapPage.java (lines 319-327)**

Empty slots are represented as null in the page's `tuples[]` array, which must stay index-aligned with the on-disk slots so `RecordId` and future inserts or deletes can address the desired slots directly. However, exposing the null state to queries that expect a tuple (such as `next()` that dereference its input immediately) can throw `NullPointerException` and also mean that empty slots are counted as rows. `HeapPage.iterator()` (line 319 – 327) prevents this null leakage by using `isSlotUsed()` during byte parsing and adding only live slots to iterated list so that null never reaches a caller.

Changes / Adaptations to API

We did not modify any method signatures in the provided API. One adaptation is in `HeapFile.readPage` (lines 86-92) which uses `RandomAccessFile`, whose I/O throws a checked `IOException`, but the `DbFile.readPage` interface does not declare throws `IOException`. We therefore convert it to an unchecked exception to conform to the fixed interface (lines 101-103)

Missing / Incomplete Elements

The write-path methods in `BufferPool`, `HeapFile`, and `HeapPage` (`insertTuple`, `deleteTuple`, `wriePage`, `markDirty`, `markSlotUsed`) and `BufferPool` eviction (`evictPage`) remain as provided stubs. These were intentionally unimplemented, and will be completed in future labs. For Lab 1, when cache is full, `getPage()` throws rather than evicting.